Deploying a Three-Tier Architecture Application on AWS EKS using Jenkins, Argo CD, Prometheus & Grafana

---

1. Introduction

Cloud computing enables organizations to deploy and manage applications efficiently without maintaining physical infrastructure. Modern applications require automation, scalability, security, and continuous monitoring to ensure reliable performance. DevSecOps combines Development, Security, and Operations into a single workflow, allowing applications to be built, tested, secured, and deployed automatically.

This project demonstrates the deployment of a Three-Tier Architecture application on Amazon EKS using Terraform, Jenkins, Docker, Kubernetes, Argo CD, SonarQube, Trivy, Prometheus, and Grafana. The complete deployment process is automated using CI/CD and GitOps principles, resulting in a secure, scalable, and production-ready cloud environment.

---

2. Project Objectives

The main objective of this project is to automate the deployment of a Three-Tier application using modern DevSecOps tools on AWS cloud infrastructure.

The project objectives are:

Provision cloud infrastructure using Terraform.

Create a Kubernetes cluster using Amazon EKS.

Automate software delivery using Jenkins.

Build and manage Docker containers.

Store container images in Amazon ECR.

Perform code quality analysis using SonarQube.

Scan container images for vulnerabilities using Trivy.

Deploy applications automatically using Argo CD.

Monitor infrastructure and applications using Prometheus and Grafana.

Build a scalable, secure, and highly available application deployment pipeline.

---

3. Project Architecture

The application follows the Three-Tier Architecture model consisting of Presentation, Business Logic, and Database layers.

Presentation Layer

The frontend provides the graphical user interface that allows users to interact with the application.

Business Logic Layer

The backend processes requests received from the frontend, performs business logic, and communicates with the database.

Database Layer

The database stores application data securely. Persistent storage ensures that data is retained even when Kubernetes pods restart.

---

4. Technologies Used

This project uses several cloud-native and DevSecOps technologies.

AWS EC2 – Hosts the Jenkins server.

Terraform – Automates infrastructure creation.

Docker – Packages applications into containers.

Amazon ECR – Stores Docker images securely.

Amazon EKS – Runs Kubernetes workloads.

Jenkins – Automates CI/CD pipeline.

GitHub – Stores application source code.

SonarQube – Performs static code analysis.

Trivy – Scans Docker images for vulnerabilities.

Argo CD – Automates Kubernetes deployment using GitOps.

Prometheus – Collects application and cluster metrics.

Grafana – Displays monitoring dashboards.

---

5. AWS Infrastructure Setup

The first stage of the project involves preparing the AWS cloud environment. An IAM user is created with the required permissions to access AWS services. AWS CLI is configured using the IAM credentials. Terraform provisions the required infrastructure, including the Jenkins EC2 instance. The Terraform state file is stored in Amazon S3, while DynamoDB provides state locking to prevent concurrent modifications.

---

6. Jenkins Installation and Configuration

Jenkins is installed on an Amazon EC2 instance and acts as the Continuous Integration server. Required plugins are installed to integrate Jenkins with GitHub, Docker, AWS, SonarQube, and Kubernetes. Credentials and build tools are configured so that Jenkins can automatically build, test, and deploy the application whenever code changes are pushed to GitHub.

---

7. SonarQube Integration

SonarQube is integrated into the Jenkins pipeline to analyze source code quality. It detects bugs, security vulnerabilities, duplicated code, and maintainability issues. By performing static code analysis before deployment, SonarQube helps improve software reliability and ensures that only high-quality code progresses through the deployment pipeline.

---

8. Docker and Amazon ECR

The frontend and backend applications are containerized using Docker. Jenkins builds Docker images automatically and pushes them to Amazon Elastic Container Registry (ECR). Storing images in ECR ensures secure and centralized management of application containers before deployment to Kubernetes.

---

9. Amazon EKS Cluster Setup

Amazon Elastic Kubernetes Service (EKS) is used to deploy and manage the containerized application. Kubernetes resources such as Deployments, Services, Namespaces, and Ingress are configured to provide high availability, scalability, and efficient resource management. The AWS Load Balancer Controller exposes the application to external users through a load balancer.

---

10. Argo CD Deployment

Argo CD implements GitOps by continuously monitoring the GitHub repository for Kubernetes manifest changes. Whenever updates are committed, Argo CD automatically synchronizes the changes with the EKS cluster. This approach eliminates manual deployments and ensures consistency between the Git repository and the running Kubernetes environment.

---

11. Prometheus and Grafana Monitoring

Prometheus collects performance metrics from Kubernetes nodes, pods, and applications. Grafana connects to Prometheus and presents these metrics using interactive dashboards. This enables administrators to monitor CPU usage, memory utilization, pod health, network traffic, and overall cluster performance in real time.

---

12. CI/CD Pipeline Workflow

The CI/CD pipeline automates the complete software delivery process. Developers push source code to GitHub, which triggers Jenkins to build the application. Jenkins performs code quality analysis using SonarQube, scans Docker images using Trivy, builds container images, and pushes them to Amazon ECR. Argo CD then deploys the latest version of the application to Amazon EKS. Prometheus and Grafana continuously monitor the deployed application and infrastructure.

Pipeline Flow:

GitHub → Jenkins → SonarQube → Trivy → Docker Build → Amazon ECR → Argo CD → Amazon EKS → Prometheus → Grafana

---

13. Advantages

Fully automated deployment process.

Improved software quality through continuous code analysis.

Enhanced security using vulnerability scanning.

Scalable Kubernetes-based deployment.

High availability and fault tolerance.

Real-time monitoring and performance visualization.

Reduced manual effort and deployment errors.

Faster and more reliable software delivery.

---

14. Challenges Faced

During the project implementation, several challenges were encountered, including configuring AWS IAM permissions, managing Terraform state files, integrating Jenkins with AWS services, configuring SonarQube and Trivy, installing Argo CD, setting up Kubernetes networking, and configuring monitoring dashboards. These challenges were resolved through proper configuration, testing, and troubleshooting.

---

15. Conclusion

This project successfully demonstrates a complete DevSecOps implementation for deploying a Three-Tier Architecture application on Amazon EKS. By integrating Terraform, Jenkins, Docker, Amazon EKS, Argo CD, SonarQube, Trivy, Prometheus, and Grafana, the deployment process becomes automated, secure, scalable, and easy to manage. The project showcases modern cloud-native deployment practices and provides a strong foundation for real-world enterprise application deployment.